Livre Acesso

Divisão de trabalho

Backend e dados — plano resumido

Equipe HumaniTec · Instituto Livre Acesso · Outubro de 2026

10 blocos para a equipe escolher, implementar e entregar sem concentrar trabalho.

1. Estado e corte do MVP

Pronto — não refazer

- FastAPI modular, JWT/JWKS, RLS, auditoria, outbox e erros padronizados;
- Supabase de produção em São Paulo, migrations aplicadas e banco vazio;
- CRUD versionado de alunos, sem exclusão física;
- matriz 25/25 de permissões, 110 testes e cobertura acima de 89%.

MVP restante

- **Produção:** BL-01 concluído; ainda faltam CI e backup;
- **Base:** usuários, responsáveis, modalidades, turmas e instrutores;
- **Operação:** matrículas, capacidade, chamada, faltas, alertas e relatório básico;
- **Depois do piloto:** espera avançada e integração real com WhatsApp.

O front-end será iniciado por último.

Como dividir

Cada integrante escolhe um bloco `BL-01` a `BL-10`; blocos de 6–8h podem ter duas pessoas. Avisar a escolha antes de editar e repartir qualquer trabalho que ultrapasse 8h. Código, testes e documentação fazem parte do mesmo bloco.

Regras

- somente dados sintéticos; nunca versionar `.env`, token, chave ou dump;
- não editar migrations `00100` – `00400`; sempre criar outra;
- comentar motivo, risco, PII, transação e destino — não a sintaxe;
- domínio não importa FastAPI/Supabase;
- testar sucesso, falha, limite e permissão;
- sem `DELETE` físico e sem migration/deploy antes da revisão do Matheus.

2. Blocos BL-01 a BL-05

BL-01 — Homologação real e deploy do backend

CONCLUÍDO · Prioridade **P0** · Matheus · Produção homologada

- **Fazer:** concluir `livre-acesso-verify-supabase`, criar liveness/readiness e publicar FastAPI na Vercel.
- **Como:** usar somente Publishable key, configurar variáveis no painel e testar cold start, JWT e RPC.
- **Por quê:** nada real pode entrar antes do caminho Auth → API → Supabase funcionar em produção.
- **Destino:** `scripts/`, módulo `operations`, `backend/app/main.py` e configuração Vercel.
- **Pronto:** endpoints respondem, nenhuma credencial aparece e smoke test passa.

BL-02 — Usuários, papéis e bloqueios

Prioridade **P1** · 1–2 pessoas · 5–8h · Depende de BL-01

- **Fazer:** `/me`, listagem de perfis, ativação/desativação e troca de papel.
- **Como:** RPCs auditadas, paginação, motivo obrigatório e proteção do último coordenador ativo.
- **Por quê:** a coordenação não pode depender do SQL Editor para administrar acessos.
- **Destino:** `backend/app/modules/identity/`, migration nova e testes por papel.
- **Pronto:** coordenação administra; recepção/instrutor/consulta são negados corretamente.

BL-03 — Responsáveis e histórico do aluno

Prioridade **P1** · 1–2 pessoas · 5–8h · Usa o CRUD de alunos existente

- **Fazer:** múltiplos responsáveis, contato principal, parentesco e histórico cadastral seguro.
- **Como:** relação muitos-para-muitos, endpoints em `/students/{id}/guardians` e concorrência por versão.
- **Por quê:** irmãos podem compartilhar contato e um aluno pode ter mais de um responsável.
- **Destino:** módulo `students`, `student_guardians`, RPCs e auditoria.
- **Pronto:** sempre existe um principal; listas mascaram e detalhes integrais são auditados.

BL-04 — Modalidades, turmas e instrutores

Prioridade **P1** · 2 pessoas · 6–8h · Pode começar agora

- **Fazer:** CRUD de modalidades/turmas, capacidade, horários e vínculo de instrutores.
- **Como:** novo módulo `classes`, migration com RLS, RPCs e regra de instrutor acessar só suas turmas.
- **Por quê:** modalidade não é turma; matrícula e chamada dependem de uma oferta concreta.
- **Destino:** `backend/app/modules/classes/`, novas tabelas e `/api/v1/classes`.
- **Pronto:** CRUD versionado, capacidade válida e isolamento entre instrutores testado.

BL-05 — Matrículas, capacidade e ciclo

Prioridade **P1** · 2 pessoas · 6–8h · Depende de BL-04

- **Fazer:** matricular, listar, pausar, retomar, concluir e cancelar com histórico.
- **Como:** RPC com `FOR UPDATE`, índice único parcial e concorrência na última vaga.
- **Por quê:** duas pessoas não podem ocupar simultaneamente a última vaga da turma.
- **Destino:** módulo `enrollments`, tabelas/histórico e `/api/v1/enrollments`.
- **Pronto:** operação é atômica, motivo fica privado e aluno não duplica matrícula ativa.

3. Blocos BL-06 a BL-10

BL-06 — Chamada e frequência em lote

Prioridade **P1** · 2 pessoas · 6–8h · Depende de BL-05

- **Fazer:** sessão por turma/data, chamada em lote, fechamento e correção auditada.
- **Como:** uma RPC por chamada, estados presente/ausente/justificada e idempotência.
- **Por quê:** uma requisição por aluno é lenta e cara em ambiente serverless.
- **Destino:** módulo `attendance`, novas tabelas e rotas por turma.
- **Pronto:** lote inválido não grava parcialmente e instrutor só opera turma atribuída.

BL-07 — Alertas e relatório básico

Prioridade **P1** · 1–2 pessoas · 5–8h · Depende de BL-06 e regra institucional

- **Fazer:** faltas consecutivas, alerta resolvível, resumo do painel e frequência por turma.
- **Como:** calcular por aulas ocorridas, tratar justificadas conforme decisão e persistir alertas sem duplicar.
- **Por quê:** armazenar presença só tem valor se gerar acompanhamento útil.
- **Destino:** módulos `alerts` e `reporting`, RPCs agregadas e testes de cálculo.
- **Pronto:** sequência, resolução e percentuais estão corretos e relatórios não expõem CPF.

BL-08 — Lista de espera e comunicação

Prioridade **P2** · 1–2 pessoas · 6–8h · Depende de BL-03, BL-04 e BL-05

- **Fazer:** fila por turma e base de comunicados; integração externa fica opcional.
- **Como:** ordem estável, estados auditados, confirmação humana de vaga e porta de provedor com fake.
- **Por quê:** lista de espera não é estado do aluno, e vaga não deve ser concedida automaticamente.
- **Destino:** módulos `waiting_list` e `communications`, outbox e migrations.
- **Pronto:** fila é rastreável, nenhuma seleção é "mágica" e envio fake é idempotente.

BL-09 — Segurança, logs e proteção de abuso

Prioridade **P1** · 1 pessoa · 4–6h · Pode ocorrer em paralelo

- **Fazer:** logs JSON, request ID, rate limiting, CORS final e revisão de dados sensíveis.
- **Como:** não registrar corpo/token/query pessoal; usar limite durável/plataforma, não memória global.
- **Por quê:** produção precisa explicar falhas sem vazar dados e resistir a abuso.
- **Destino:** módulo `operations`, `config.py`, middlewares e testes de privacidade.
- **Pronto:** 429 tem `Retry-After`, logs não contêm PII e origem desconhecida é bloqueada.

BL-10 — CI, backup, smoke test e documentação

Prioridade **P0/P1** · 1–2 pessoas · 5–8h · Pode ocorrer em paralelo

- **Fazer:** GitHub Actions, backup/restauração, smoke pós-deploy e atualização dos guias.
- **Como:** rodar Ruff/Mypy/Pytest/parser SQL, sem acessar produção no CI comum; criar runbooks claros.
- **Por quê:** deploy verde não prova funcionamento e backup não testado não protege ninguém.
- **Destino:** `.github/workflows/`, `scripts/`, `docs/runbooks/` e README.
- **Pronto:** pipeline bloqueia falha, smoke retorna exit code e restauração possui evidência.

4. Escolha, entrega e revisão

Quadro da equipe

- Pedro da Costa Borges — `BL-__`
- Pedro Henrique de Caniza Gomes — `BL-__`
- Rafael Belgrano Paixão Costa — `BL-__`
- Giovana Oliveira de Queiroz — `BL-__`
- Sara Adriele de Lima Andrade — `BL-__`
- Júlio César Soares Cavalcante — `BL-__`
- Matheus Pinheiro Barbosa — `BL-01 concluído` + revisão/produção
- Victor Gabriel Santos Silva — `BL-__`
- Victor Augusto Ferreira Silva — `BL-__`
- Rodrigo Ferreira dos Santos Pereira — `BL-__`
- Kauã Thalles Silva de Jesus — `BL-__`

Sugestão: uma pessoa em BL-01, BL-02 e BL-03; duplas em BL-04, BL-05 e BL-06; uma pessoa em BL-09/BL-10. BL-07 e BL-08 começam depois das dependências.

Checklist curto

- [ ] avisei o bloco e mantive seu escopo;
- [ ] usei somente dados sintéticos e migration nova;
- [ ] comentei decisões/riscos e testei permissões;
- [ ] rodei Ruff, Mypy e Pytest;
- [ ] criei commits pequenos em português;
- [ ] pedi revisão do Matheus.

Branch ou `main`

Preferência: criar `feat/bl-XX-nome`, após `git pull`. Também **pode codar diretamente na** `main`: avisar antes, evitar arquivos já assumidos, separar commits e nunca aplicar migration/deploy sozinho.

Commits curtos: `feat: cria turmas`, `fix: limita vagas`, `test: valida chamada` ou `chore: documenta frequência`.

Em branch ou na `main`, **Matheus Pinheiro Barbosa fará a revisão final** antes de encerrar a entrega e antes de qualquer migration, deploy ou dado real.

Objetivo: blocos pequenos, compreensíveis, testados e seguros — não quantidade de código.